

Test Suite Classification and Benchmarking

The test suite is designed to systematically validate both **functional correctness** and **runtime behavior** of the virtual machine. Tests progress from basic instruction validation to complex recursive and nested workloads that stress stack discipline, call-frame management, and control flow. Instruction counts and execution times are recorded for representative programs to provide basic performance insights, as required by the assignment.

1 Test Classification by Functionality

1.1 Arithmetic Expressions & Stack Primitives

These tests validate core stack behavior and arithmetic correctness.

- **Tests:** T1–T8, T10, T15, T25
- **Features Tested:**
 - PUSH, ADD, SUB, MUL, DIV
 - DUP, SWAP, DROP, OVER
 - Unary ops (INC, DEC, NEG)
 - Comparisons (EQ, LT, GT)
- **Instruction Count Range:** 3–9
- **Typical Runtime:** $\sim 3\text{--}4\ \mu\text{s}$
- **Purpose:** Establish correctness of fundamental stack semantics

1.2 I/O and Miscellaneous Instructions

Tests that validate side-effecting and no-op instructions.

- **Tests:** T9 (PRINT), T10 (NOP)
- **Observation:** PRINT has noticeably higher execution time due to I/O
- **Example:**
 - T9: 3 instructions, $\sim 66\ \mu\text{s}$

1.3 Loops and Branching

These tests validate conditional and unconditional control flow.

- **Tests:** T11, T13, T14, T23
- **Patterns:**
 - Counter loops
 - Conditional branching
 - Loop termination via JZ
- **Instruction Count Range:** $\sim 26\text{--}43$
- **Typical Runtime:** $\sim 4\ \mu s$
- **Purpose:** Verify correct PC manipulation and branching behavior

1.4 Global Memory Load/Store

Tests correctness of the global data segment.

- **Tests:** T12, T23
- **Features Tested:**
 - STORE and LOAD instructions
 - Persistence of values across control flow
- **Instruction Count:** $\sim 8\text{--}43$
- **Observation:** Memory access cost is negligible compared to control flow

1.5 Simple Function Calls

Tests basic CALL/RET mechanics without arguments or nesting.

- **Tests:** T16, T17, T24
- **Features Tested:**
 - Frame setup and teardown
 - Return value propagation
- **Instruction Count:** $\sim 4\text{--}8$
- **Runtime:** $\sim 60\text{--}65\ \mu s$
- **Observation:** CALL/RET overhead dominates runtime for small programs

1.6 Nested Function Calls

Validate correctness under nested activation records.

- **Tests:** T18, T22
- **Features Tested:**
 - Multiple active stack frames
 - Correct return address restoration
- **Instruction Count:** $\sim 6\text{--}8$
- **Runtime:** $\sim 80\ \mu s$
- **Purpose:** Stress frame-pointer correctness

1.7 Iterative Computation Using Calls

Tests iterative behavior implemented using function calls.

- **Tests:** T19, T20
- **Examples:**
 - Argument passing via `LOAD_LOCAL`
 - Looping via repeated calls
- **Instruction Count:** $\sim 7\text{--}25$
- **Runtime:** $\sim 77\text{--}89\ \mu s$

1.8 Recursive Computation (Moderate Depth)

Tests correctness of recursion and argument access.

- **Tests:** T21 (Factorial), T32 (Tribonacci), T33 (Power), T35 (Sum of squares)
- **Instruction Count Range:** $\sim 63\text{--}205$
- **Runtime Range:** $\sim 167\text{--}193\ \mu s$
- **Purpose:** Validate frame cleanup and recursive returns

1.9 Stress Tests: Deep Recursion

These tests intentionally stress stack depth and frame management.

- **Tests:** T26 (Fibonacci), T34 (Ackermann)
- **Instruction Count:**
 - Fibonacci ($n=6$): ~ 317 instructions
 - Ackermann(2,2): ~ 292 instructions
- **Observation:** Recursive branching causes rapid instruction growth
- **Purpose:** Expose frame corruption and stack exhaustion bugs

1.10 Stress Tests: Deeply Nested Loops

Tests extreme control-flow nesting without recursion.

- **Tests:** T28 (2-level), T29 (3-level), T30 (5-level), T31 (10-level)
- **Instruction Count Growth:**
 - T28: ~195 instructions
 - T29: ~651 instructions
 - T30: ~880 instructions
 - T31: ~28,661 instructions
- **Runtime:**
 - T31: ~41 ms
- **Purpose:** Stress PC updates, local-variable access, and loop overhead

2 Benchmarking Summary

- Execution time scales **roughly linearly with instruction count**
- CALL-heavy programs are slower than arithmetic-heavy programs
- Recursive programs exhibit exponential instruction growth
- Deeply nested loops stress control-flow handling more than recursion
- Unified stack design increases overhead for function-heavy workloads

These results demonstrate predictable runtime behavior and provide sufficient comparative benchmarks across program sizes, as required by the assignment.